Конспект: Алгоритмы и структуры данных, лекции 2–6

Oct 26, 2026 · @Sokrat

Как читать этот конспект

Пять лекций складываются в одну линию: как C-код превращается в программу → как на C писать → как проверять, что код работает → сколько памяти занимают данные → как из этого собрать динамический массив и список. В конце курса по лабам вы пишете именно `Vector` и `List`, поэтому лекции 4–6 — самые важные.

Лекция	Тема	Главное, что нужно унести
2	Компилятор и интерпретатор	4 этапа сборки gcc, зачем Makefile, как читать ошибки
3	Синтаксис C	функции, область видимости, `malloc/free`, строки
4	Тесты	`assert`, функциональные / граничные / стресс-тесты
5	Базовые типы	`sizeof` каждого типа, указатели, выравнивание структур
6	Динамические структуры	`Vector` на `void*`, связный список, асимптотика

Файл «л1_2 1.pdf» — точная копия «л1_2.pdf», и внутри у него титул «Лекция 2». Отдельного файла для лекции 1 в папке нет.

Лекция 2. Компилятор, интерпретатор и сборка C-программы

Идея лекции: компьютер понимает только машинный код, и кто-то должен перевести ваш текст на C в него. В C это делает компилятор заранее, за 4 этапа.

Транслятор = компилятор или интерпретатор

Транслятор — программа, которая переводит код с одного языка на другой. Бывает двух видов:

- **Компилятор** (C, Assembler) — переводит всю программу целиком до запуска и выдаёт исполняемый файл. Как переводчик, который перевёл книгу и отдал готовый текст.
- **Интерпретатор** (Python, JavaScript) — читает и выполняет код по ходу работы. Как синхронный переводчик: услышал фразу — перевёл.

У Python много интерпретаторов. Стандартный — **CPython**: он сначала компилирует код в байткод (`.pyc`), потом выполняет его в виртуальной машине. Медленный, нет JIT, есть ограничение GIL (одновременно выполняется один поток). Альтернативы: PyPy (JIT, быстрее в 2–5 раз), Jython (на JVM), IronPython (на .NET), MicroPython (для микроконтроллеров), Brython (Python → JavaScript в браузере).

4 этапа сборки в gcc

Команда `gcc -Wall -save-temps test.c -o test` оставляет промежуточные файлы всех этапов — можно посмотреть глазами.

Этап	Что делает	Результат	Команда
1. Препроцессинг	вставляет содержимое `#include`, подставляет `#define`, удаляет комментарии	`.i` — всё ещё текст на C	`gcc -E main.c -o main.i`
2. Компиляция	разбирает код, строит AST, оптимизирует (`-O0` … `-O3`)	`.s` — текст на ассемблере	`gcc -S main.i -o main.s`
3. Ассемблирование	переводит ассемблер в машинный код, добавляет таблицу символов	`.o` — объектный файл, ещё не запускается	`gcc -c main.s -o main.o`
4. Линковка	склеивает `.o` и библиотеки, находит адреса функций	исполняемый файл (`a.out`, `.exe`)	`gcc main.o -o main`

Что важно понять про каждый этап:

- **Препроцессор — это просто замена текста.** `#define LAST 10` буквально заменяет все `LAST` на `10`. `#include <stdio.h>` вставляет весь файл stdio.h в ваш код.
- **Компиляция** внутри тоже идёт по шагам. Сначала **лексический анализ**: текст режется на лексемы. `a = 2;` → `ID(a) ASSIGN INTVAL(2) SEMICOLON`. Потом **парсинг**: из лексем строится дерево AST (Abstract Syntax Tree), где операции — узлы, а операнды — дети. Из дерева генерируется ассемблер.
- **Объектный файл** `.o` ещё нельзя запустить: в нём есть «дыры» — вызовы функций вроде `printf`, адрес которых пока неизвестен.
- **Линкер** закрывает эти дыры. Библиотеки подключаются двумя способами. **Статически** (`.a`, `.lib`) — код вшивается в программу: файл больше, но автономный. **Динамически** (`.so`, `.dylib`, `.dll`) — подгружается при запуске: файл меньше.
- Если в программе нет `main`, ошибка будет именно на линковке: `Undefined symbols: "_main"` — линкер не нашёл точку входа.

Компиляторы и история

C создал Dennis Ritchie в 1972 году для написания UNIX. Первый стандарт ANSI C (C89) — 1989. C++ создал Bjarne Stroustrup в 1983 как «C with Classes», стандарт C++98.

Основные компиляторы: **GCC** (Linux, кроссплатформенный), **Clang** (часть LLVM, на macOS, быстрый и с понятными ошибками), **MSVC** (Windows, Visual Studio). Ещё бывают Intel ICC (для HPC), TinyCC (маленький, без оптимизаций).

Установка: Linux — `sudo apt install build-essential`, Mac — `brew install gcc` (или уже есть clang). Проверка: `gcc --version`. Сборка и запуск: `gcc test.c -o test`, потом `./test`.

Makefile

Чтобы не набирать длинную команду gcc каждый раз, её записывают в Makefile. Дальше достаточно `make` (собрать), `make run` (собрать и запустить), `make clean` (удалить бинарник).

```
TARGET = test               # имя исполняемого файла
CC     = gcc                # компилятор
CFLAGS = -Wall -Wextra -O2  # флаги: все предупреждения + оптимизация
SRC    = test.c

all: $(TARGET)              # цель по умолчанию

$(TARGET): $(SRC)           # чтобы собрать test, нужен test.c
        $(CC) $(CFLAGS) $(SRC) -o $(TARGET)

run: $(TARGET)
        ./$(TARGET)

clean:
        rm -f $(TARGET)
```

Структура правила: `цель: зависимости`, на следующей строке через **таб** — команда. Другие системы сборки: CMake (стандарт для больших проектов, генерирует Makefile), Ninja (очень быстрая), Meson, Bazel (от Google).

Как читать ошибки

- `make: *** No rule to make target 'test1.c'` — ошибка в Makefile: указан файл, которого нет.
- `test.c:6:19: error: expected ';'` — ошибка в коде. Числа — это **строка:столбец**, сразу идите туда.
- `use of undeclared identifier 'i'` — переменная используется вне своей области видимости. Например, `i` объявлена в `for (int i = ...)`, а вы печатаете её после цикла.

В VS Code стоит поставить расширение C/C++ — оно нужно в основном для отладки (дебага) и подсветки ошибок.

Лекция 3. Основы синтаксиса C

Идея лекции: в C вы сами отвечаете за типы и за память. Всё остальное (условия, циклы) похоже на другие языки.

Функции и main

Шаблон функции: `тип_результата имя(тип аргумента, ...) { ... return значение; }`. Тип указывается и у аргументов, и у результата.

```
int sum2nums(int num1, int num2) { return num1 + num2; }
```

`main` — **точка входа**, с неё начинается программа. Она обязательна и возвращает `int` — код завершения: `0` — успех, `1` — ошибка, другие числа — ваши собственные коды ошибок. Хороший `main` делает три вещи: читает вход (`scanf("%d", &n)`), вызывает логику, печатает результат.

Пример со слайда — округление до `dec` знаков. Умножаем на 10^dec, отбрасываем дробную часть через `(int)`, делим обратно:

```
float round_float(double num, int dec) {
    int k = pow(10, dec);          // 2 знака -> 100
    int rounded = (int)(num * k);  // 1.2345 * 100 = 123.45 -> 123
    return (float)rounded / k;     // 123 / 100 = 1.23
}
```

Строго говоря, это не округление, а отбрасывание: 1.239 даст 1.23, а не 1.24.

Переменные и константы

- `const int x = 1;` — константа, присвоить заново нельзя (ошибка компиляции).
- `bool` доступен со стандарта C99 через `#include <stdbool.h>`.
- **Два способа задать строку — и они разные:**
  - `char* s = "Hello";` — указатель на строковую константу. Она лежит в области «только для чтения», поэтому `s[0] = 'h'` скомпилируется, но на запуске обычно упадёт.
  - `char s[] = "Hello";` — массив на стеке, копия строки. `s[0] = 'w'` — нормально.
- `s[10] = 'w'` при длине 6 — выход за границу. **Компилятор C границы массивов не проверяет**, поэтому ошибки нет. Дальше три исхода: вы перезапишете соседнюю переменную («ниндзя-баг»), перезапишете мусор (ничего не случится) или выйдете за пределы стека (программа упадёт).

Область видимости

Переменная живёт только внутри фигурных скобок `{}`, где объявлена. Есть три вида памяти:

Вид	Что там	Сколько живёт
Локальные переменные (стек)	всё, что объявлено внутри функции или блока	до закрывающей `}`
Динамическая память (куча)	то, что выделено через `malloc`	пока вы не вызовете `free`
Глобальные переменные	объявленные вне функций, и строковые константы	всю программу

Задачка со слайда. В функции `scope_demo(int arg)` объявлены `i` в начале, `j` внутри `if`, `k` внутри `for`, `w` внутри `else`. Перед `return` доступны только `arg` и `i`.

А если внутри `if` написать `int *arr = malloc(...)`? Сама переменная-указатель `arr` исчезнет на `}`, а выделенная память останется. Адрес потерян, освободить её уже нельзя — это **утечка памяти**.

Условия

В C любое число, кроме 0, — это «истина». Разбор упражнения со слайда:

Условие	Сработает?	Почему
`if (0)`, `if (false)`, `if (0.0)`	нет	ноль = ложь
`if (1)`, `if (-1)`, `if (true)`, `if (1.1)`	да	не ноль = истина, знак не важен
`if ('a')`	да	символ — это число (код 97)
`if (1 || 0)`	да	ИЛИ
`if (1 && 0)`	нет	И
`if (1 ^ 0)`	да	`^` — исключающее ИЛИ по битам, 1 ^ 0 = 1

switch и проваливание (fall-through). После подходящего `case` код выполняется дальше вниз, пока не встретит `break`. В упражнении с `color = 2` напечатается `color 2`, потом `color 3`, потом `NA`, и функция вернёт `-1`. Всё потому, что в `case 2` и `case 3` нет `break`. Так же можно специально объединять несколько case с одним действием.

Циклы

- `for (инициализация; условие; шаг)` — когда знаем число итераций.
- `while (условие) {}` — сначала проверяем, потом делаем. Может не выполниться ни разу.
- `do {} while (условие);` — сначала делаем, потом проверяем. Выполнится минимум один раз.

Задача про пароль (вводить символы до первой ошибки) естественнее решается через `do-while`: символ нужно сначала прочитать, и только потом сравнивать. В варианте с `while` на слайде переменная `input_sym` сравнивается до того, как в неё что-то записали.

Библиотеки и заголовочные файлы

Библиотека	Для чего
`<stdio.h>`	`printf`, `scanf`, работа с файлами
`<stdlib.h>`	`malloc`, `free`, `atoi`
`<string.h>`	`strcmp`, `strcpy`, `strlen`
`<math.h>`	`pow`, `sqrt`
`<stdbool.h>`	`bool`, `true`, `false`

`#include <...>` ищет в системных папках, `#include "mytools.h"` — сначала в папке проекта.

Код делят на два файла. **`.h` (заголовочный)** — «что есть»: объявления структур, сигнатуры функций без тела, константы. **`.c`** — «как работает»: реализации функций.

```
// color.h
struct Color { unsigned char red, green, blue; };
struct Color* removeRedChannel(struct Color* color);

// color.c
struct Color* removeRedChannel(struct Color* color) {
    struct Color* copy = malloc(sizeof(struct Color));
    if (copy != NULL) {
        *copy = *color;   // копируем всю структуру
        copy->red = 0;    // -> доступ к полю через указатель
    }
    return copy;
}
```

Динамическая память

Функция	Что делает
`malloc(5 * sizeof(int))`	выделяет память под 5 int. Внутри мусор
`calloc(5, sizeof(int))`	то же, но заполняет нулями
`realloc(arr, 10 * sizeof(int))`	расширяет до 10 элементов, старые данные сохраняются
`free(arr)`	освобождает память

`malloc` просто резервирует кусок памяти подряд и возвращает адрес его начала. `arr + 2` — адрес третьего элемента. Всегда проверяйте, что `malloc` не вернул `NULL`, и на каждый `malloc` делайте ровно один `free`.

Важная задача со слайда. Почему из функции можно вернуть `return "blue";` и всё работает? Строковые константы лежат не на стеке, а в отдельной read-only области и живут всю программу. А вот вернуть локальный массив `char buf[10]` нельзя — он исчезнет при выходе из функции.

Функции для строк (`<string.h>`)

Строка в C — это массив `char`, который заканчивается символом `'\0'`. По нему все функции понимают, где конец.

Функция	Что делает
`strlen(s)`	длина без `\0`

Функция	Что делает
`strcpy(dst, src)` / `strncpy(dst, src, n)`	копирует строку (не больше n символов)
`strcat(dst, src)` / `strncat`	дописывает в конец
`strcmp(a, b)`	сравнивает, `0` = равны
`strchr` / `strrchr`	первое / последнее вхождение символа
`strstr(s, "abc")`	поиск подстроки
`strdup(s)`	копия строки в куче (потом нужен `free`)
`strtok(s, ",")`	режет строку на куски по разделителю
`sprintf` / `snprintf(buf, size, ...)`	печатает в строку (вторая — с ограничением размера, безопаснее)
`sscanf("42", "%d", &x)`	читает данные из строки
`memset` / `memcpy` / `memcmp`	заполнить / скопировать / сравнить блок байтов

Организация курса

Перед вопросом ментору ищите ответ в репозитории. Вопрос задаётся в Discussions на GitHub, ссылка — ментору в личку. Статусы проверки лабы:

- **Change request** — нужны правки, лаба уходит в конец очереди.
- **Approve** — предварительно принято, нужна очная защита.
- **Merge** — лаба сдана, баллы стоят. Если ментор сразу смёрджил, на защиту идти не нужно.

Лекция 4. Тесты

Идея лекции: тест — это пара «вход → ожидаемый выход», которую проверяет программа, а не вы глазами. Самые ценные тесты — граничные: именно там чаще всего ломается код.

Три способа проверить функцию

1. **Через `scanf`** — каждый раз вводите данные руками и смотрите на вывод. Медленно.

2. **Через запуски** — захардкодите значения в коде, запускаете, смотрите на `printf`. Уже лучше, но всё ещё глазами.
3. **Через тесты** — `assert(calc(2, 1, '+') == 3);`. Если условие ложно, программа падает и пишет, какая строка не прошла. Запускается одной командой.

Зачем тесты: быстрее ручной проверки, ловят ошибки рано, показывают, что новое изменение не сломало старое, и не устают. Но это не панацея: если требования меняются каждый день, тщательные тесты быстро устаревают.

Уровни тестов (пирамида)

- **Unit** — проверяет одну маленькую независимую единицу, обычно одну функцию. Их больше всего.
- **Компонентные** — логический блок из нескольких функций (например, авторизация).
- **Интеграционные** — взаимодействие нескольких компонентов.
- **End-to-End** — целый пользовательский сценарий. Их меньше всего.

Виды тестов на курсе

Вид	Что проверяет	Пример
Функциональные	решает ли алгоритм задачу в обычных случаях	сортировка [3,1,2] → [1,2,3]
Граничные	крайние случаи: пустой вход, один элемент, максимум типа, ноль, NULL	`[]`, `[5]`
Стресс-тесты	случайные или худшие данные, время	QuickSort на уже отсортированном массиве
Эффективность (позже)	время на входах разного размера	10^3, 10^5, 10^7 элементов

Хороший тест: автоматизируемый, повторяемый, изолированный (один тест — одна проверка) и понятный.

Как тесты организованы в лабах

Три файла: `vector.h` (объявления), `vector.c` (реализация) и `tests.c`. В `tests.c` подключаются `<assert.h>` и `"vector.h"`, а в `main` идут блоки `assert`, сгруппированные по виду тестов.

Пример 1: калькулятор

Функция `float calc(short a, short b, char symbol)` делает + − × / и возвращает `-1` при неверном знаке.

- **Функциональные:** по одному тесту на каждый знак: `2 + 1 = 3`, `2 − 1 = 1`, `2 × 1 = 2`, `2 / 1 = 2`.
- **Граничные:** `short` хранит от −32768 до 32767, поэтому `32767 + 1 = 32768`, `−32768 − 1 = −32769`, `32767 × 32767 = 1 073 676 289`.
- **Деление:** `5 / 0 → -1`, `0 / 5 = 0`, `−10 / 5 = −2`.
- **Неверный знак:** `'@'`, `'\0'` → `-1`.
- **Нули:** `0 + 0`, `0 − 0`, `0 × 0`, `0 / 0`.

Тесты нашли две ошибки: деление на ноль не обрабатывалось, а точности `float` не хватает для `1 073 676 289` (у float около 7 точных цифр). Ещё одна тонкость: `-1` как код ошибки неотличим от честного результата `1 − 2 = −1`.

Пример 2: подсчёт делителей числа

Первая версия перебирает все `i` от 1 до `num`. Функциональные тесты проходят (6 → 4, 5 → 2, 100 → 9), граничные тоже (0 → 0, 1 → 1). Но стресс-тест на 2 147 483 647 считается 4 секунды и не укладывается в лимит 1 секунда. И для отрицательных чисел цикл вообще не выполняется.

Исправление — идти только до √num. Делители идут парами: если `i` делит `num`, то и `num / i` тоже. Поэтому каждый найденный `i` даёт +2. Исключение — когда `i × i = num` (10 для 100), тогда +1. Для отрицательных чисел считаем для модуля и умножаем на 2 (делители со знаком минус). Сложность падает с O(N) до O(√N): для 2·10⁹ это около 46 000 шагов вместо 2 миллиардов.

```
int countDivisorsOfNumber(int num) {
    size_t limit = sqrt(num >= 0 ? num : -num);
    int count = 0;
    for (int i = 1; i <= limit; i++) {
        if (num % i == 0) count += (i * i == num) ? 1 : 2;
    }
    if (num < 0) count *= 2;
    return count;
}
```

Тест на время делается через `clock()` из `<time.h>`:

```
clock_t start = clock();
countDivisorsOfNumber(2147483647);
double seconds = (double)(clock() - start) / CLOCKS_PER_SEC;
assert(seconds < 1.0);
```

Примечание: для −6 на слайде написано и «4», и «8 делителей». В исправленном коде и тестах правильный ответ — 8 (±1, ±2, ±3, ±6).

Тесты для динамического массива (спойлер к лабе)

Это почти готовый список того, что стоит проверить в своей лабе.

Функция	Функциональные	Граничные
`vectorCreate(elem_size)`	создать для `int`, `float` / `double`, `char`, `bool`, `char*`, `Vector*`	`elem_size == 0`
`vectorAppend(v, el)`	добавить 1 элемент; добавить 2–7, чтобы проверить рост `capacity` и `size`	`NULL` вместо вектора или значения; значение другого типа
`vectorGet(v, index)`	append, потом get и сравнить; много append (рост capacity) и проверить все; рост, потом удаление и проверка	пустой вектор; `NULL`; индекс за `size` и за `capacity`; последний после append + remove
`vectorSet(v, index, value)`	заменить один; заменить все после роста и после уменьшения capacity	пустой вектор; `NULL`; отрицательный индекс; индекс 0; индекс = size; за size
`vectorRemove(v, el)`	домашнее задание со слайда — придумайте сами по той же схеме	

Вопрос со слайда «это всё ещё unit-тест?» про `vectorGet`: тест пользуется и `append`, и `get`, то есть проверяет сразу несколько функций. Строго говоря, это уже ближе к компонентному тесту.

Лекция 5. Базовые (статичные) структуры данных

Идея лекции: у каждого типа есть точный размер в байтах, и от выбора типа зависит, сколько памяти займёт программа. Пример со слайда: датасет шахматных партий занимает 7,21 ГБ, если брать типы не думая (int64), и 3,90 ГБ, если подобрать подходящие (int16 и т. п.).

Почему C, а не Python

Потому что в Python всё спрятано. `x in my_set` выглядит как одна операция, но в исходниках CPython (`setobject.c`) внутри всё равно цикл. В C вы пишете эти циклы сами и видите цену каждой операции. К тому же в Python любое число — это целый объект: массив из миллиона int весит примерно в 5 раз больше, чем в C, а из миллиона double — примерно в 4 раза.

Размеры типов

Тип	Байт	Диапазон
`bool` / `_Bool`	1	0 или 1
`char` , `int8_t`	1	−128 … 127
`unsigned char` , `uint8_t`	1	0 … 255
`short int`	2	−32 768 … 32 767
`unsigned short`	2	0 … 65 535
`int`	4	−2 147 483 648 … 2 147 483 647
`unsigned int`	4	0 … 4 294 967 295
`long int` , `long long`	8	-2^{63} … $2^{63}-1$
`float`	4	$\pm 3{,}4\cdot 10^{38}$, около 7 точных цифр
`double`	8	$\pm 1{,}8\cdot 10^{308}$, около 15–16 точных цифр
`long double`	16	$\pm 1{,}2\cdot 10^{4932}$
любой указатель (`int*` , `bool*` , `void*`)	8	адрес в памяти

Размеры `long` и указателей даны для 64-битной системы (Mac, Linux). Диапазон легко посчитать: n бит дают 2^n значений. 1 байт = 8 бит = 256 значений: либо 0…255, либо −128…127.

Разбор загадок с sizeof

Выражение	Результат	Почему
`sizeof(true)`	4	`true` — это макрос, заменяется на число `1`, а число имеет тип `int`
`sizeof(bool)`, переменная `bool`	1	сам тип bool занимает 1 байт
`bool arr[10]`	10	10 × 1 байт
`bool* arr = malloc(10)`	8	`sizeof` указателя — размер адреса, а не массива
`sizeof(2147483647)`	4	влезает в `int`
`sizeof(2147483648)`	8	на 1 больше максимума int → компилятор берёт `long`
`int arr[10]`	40	10 × 4
`sizeof('a')`	4	в C символьный литерал имеет тип `int` (в C++ было бы 1)
`char c = 'a'; sizeof(c)`	1	переменная типа `char`
`sizeof("a")`	2	символ + `'\0'` в конце
`sizeof("ы")`	3	русская буква в UTF-8 занимает 2 байта + `'\0'`
`char s[] = "hello"`	6	5 + `'\0'`
`char s[] = "Привет"`	13	6 букв × 2 байта + `'\0'`

Главный вывод: **`strlen("Привет")` вернёт 12, а не 6** — функции строк считают байты, а не буквы.

float и double внутри (стандарт IEEE 754)

Дробное число хранится как «научная запись» в двоичной системе: знак, порядок (экспонента) и цифры (мантисса).

- `float` = 32 бита: 1 знак + 8 экспонента + 23 мантисса.
- `double` = 64 бита: 1 знак + 11 экспонента + 52 мантисса.

```
\text{value} = (-1)^{\text{sign}} \cdot (1 + \text{fraction}) \cdot 2^{\text{exponent} - 127}
```

Пример для 100: sign = 0, exponent = 133 (133 − 127 = 6), fraction = 0,5625. Итого (1 + 0,5625) × 2^6 = 1,5625 × 64 = 100. А вот 1010,1097 в float превращается в 1010,109680: битов мантиссы не хватает, и число хранится приближённо. Отсюда правило: **float нельзя сравнивать через `==`**. Сравнивайте с допуском: `fabs(a - b) < 1e-6`.

Указатели

Указатель — это переменная, в которой лежит адрес другой переменной. Размер у любого указателя одинаковый — 8 байт.

- `&x` — взять адрес переменной.
- `*p` — взять значение по адресу (разыменование).
- `p + 1` — адрес следующего элемента. Сдвиг идёт не на 1 байт, а на `sizeof(тип)`. Для `short*` — на 2 байта, для `int*` — на 4.
- `*(arr + i)` — то же самое, что `arr[i]`.

На слайде две `short` переменные лежат по адресам `...e06` и `...e04`. Вторая — на 2 байта *ниже*, потому что стек растёт от старших адресов к младшим. Поэтому `var1 - 1` указывает ровно на `val2`.

В куче (`malloc`) соседние элементы лежат по возрастанию: `arr` = `...070`, `arr + 1` = `...072`. Из схемы видно, что один `short` — это два байта, которые читаются как одно число: байты 1 и 2 дают 2 × 256 + 1 = 513.

Структуры и выравнивание

Размер структуры — не просто сумма полей. Процессор быстрее читает `int`, если тот лежит по адресу, кратному 4, а `double` — кратному 8. Поэтому компилятор вставляет пустые байты (padding).

Структура	sizeof	Раскладка по байтам
`struct A { int x; }`	4	int (4)
`struct B { char c; int x; }`	8	char (1) + пусто (3) + int (4)
`struct C { int x; char c; }`	8	int (4) + char (1) + пусто (3) — хвост добивается, чтобы в массиве структур следующий int тоже был выровнен
`struct D { char c; double d; }`	16	char (1) + пусто (7) + double (8)

Совет: объявляйте поля от больших к меньшим — так пустых байтов меньше.

Заготовка к лабе — структура вектора (подробно в лекции 6):

```
typedef struct {
    void*  data;       // данные
    size_t size;       // сколько элементов сейчас
    size_t elem_size;  // размер одного элемента в байтах
    size_t capacity;   // на сколько элементов выделена память
} Vector;
```

Упражнения на указатели перед 3-й лабой

Базовые:

- [ ] 1. Указатель на `int`: вывести адрес и значение через указатель
- [ ] 2. Обменять две переменные через указатели
- [ ] 3. Функция принимает `int*` и увеличивает значение на 1
- [ ] 4. Пройти `int[5]` указателем и вывести элементы
- [ ] 5. Свой `strlen` на указателях

Средние:

- [ ] 6. `swap` двух элементов массива
- [ ] 7. Копирование строки `src → dst` указателями
- [ ] 8. Минимум массива через указатель
- [ ] 9. Вывести строку задом наперёд, двигаясь от конца
- [ ] 10. Массив указателей на строки и их вывод

Продвинутые:

- [ ] 11. Функция принимает `int**` и выделяет внутри массив («возврат массива»)
- [ ] 12. Динамический двумерный массив `int**`
- [ ] 13. Сортировка массива строк `char**`
- [ ] 14. Свой `strstr` на указателях
- [ ] 15. Структура с полем-указателем: выделить и освободить память под поле

Лекция 6. Динамические структуры: массив и список

Идея лекции: вместо того чтобы каждый раз вручную делать `malloc` / `realloc` и отдельно таскать размер, мы заворачиваем это в структуру `Vector`. А чтобы она работала с любым типом, храним данные через `void*`. Всё это показано на одной задаче — сглаживании временного ряда.

Массив или список: что когда брать

- **Массив** — элементы лежат в памяти подряд. Любой элемент по индексу получаем сразу: адрес = начало + i × размер. Пример задачи: температура по регионам, где нужно часто читать и менять значение по номеру.
- **Связный список** — каждый элемент (узел) лежит где угодно и хранит указатель на следующий. Чтобы добраться до i-го, надо пройти все предыдущие. Зато добавить или убрать элемент с края — просто перевести указатель. Пример задачи: очередь задач, куда добавляем в конец и забираем с обоих концов.

Асимптотика

O(1) — время не зависит от размера. O(N) — растёт пропорционально числу элементов. O*(1) — **амортизированно** O(1): обычно быстро, но изредка дорого, а в среднем выходит O(1).

Операция	Массив	Односвязный список	Двусвязный список
доступ `[i]`	O(1)	O(N)	O(N)
первый / последний	O(1)	O(1)	O(1)
`append`	O*(1)	O(1)	O(1)
`set(i, value)`	O(1)	O(N)	O(N)
`pop(0)`	O(N)	O(1)	O(1)
`pop(-1)`	O*(1)	O(N)	O(1)
`pop(i)`	O(N)	O(N)	O(N)
`find(value)`	O(N)	O(N)	O(N)
память	N × размер + заголовок	N × 16 + 8 байт	N × 24 + 8 байт

Как читать таблицу:

- `pop(0)` у массива — O(N), потому что все остальные элементы надо сдвинуть влево.
- `pop(-1)` у односвязного списка — O(N): чтобы удалить последний, нужно найти предпоследний, а указателей назад нет. В двусвязном есть `prev`, поэтому O(1).

- Узел односвязного списка = 16 байт (указатель на данные + `next`), двусвязного = 24 (ещё `prev`). Список всегда тяжелее массива.

Почему append у массива амортизированно O(1). У вектора два разных числа: `size` (сколько элементов есть) и `capacity` (на сколько места выделено). Пока `size < capacity`, добавление — просто запись в свободную ячейку. Когда место кончилось, `realloc` выделяет новый блок и копирует туда всё — это O(N). Если при этом увеличивать `capacity` вдвое, дорогие копирования случаются всё реже, и в среднем на один append выходит O(1). А вот `realloc` на +1 элемент каждый раз (как в первой версии кода на лекции) даёт O(N) на каждое добавление.

Задача: сглаживание скользящим средним

На входе файл с числами через пробел. Нужно заменить каждое число средним по «окну» из `window_size` соседей и записать результат в файл. Пример с окном 3: `1 2 3 4 5` → средние (1,2,3), (2,3,4), (3,4,5) → `2 3 4`. Выход короче входа на `window_size - 1`.

Порядок работы, который советует лектор: псевдокод → подумать → реализация → подумать → придумать тесты → подумать → написать тесты.

Шаг 1. Чтение файла `int8_t* read_array(char* filename, size_t* arr_size)`. Логика:

1. Открыть файл `fopen(filename, "r")`, проверить на `NULL`.
2. Читать по одному символу `fgetc(file)`, пока не `EOF`.
3. Если символ — не пробел, дописать его в буфер `digit_str`.
4. Если пробел и буфер не пустой — превратить буфер в число `strtol(digit_str, &end, 10)`, добавить в массив, очистить буфер (`digit_str[0] = '\0'`).
5. После цикла — **не забыть последнее число**: после него нет пробела, оно всё ещё в буфере.
6. Закрыть файл `fclose`.

Вопросы со слайдов и ответы на них:

- *Почему* `digit_str[5]`*?* Число `int8_t` — от −128 до 127, максимум 4 символа (`-128`) + `'\0'`.
- *Почему* `malloc((arr_size + 1) * ...)` *при* `arr_size = 0`*?* Чтобы сразу было место под первый элемент.

- *Как узнать размер возвращённого массива?* Никак — указатель не знает длину. Поэтому размер возвращают через дополнительный аргумент-указатель `size_t* arr_size`.
- *Где может сломаться `strtol`?* На чужеродных символах (`"a"` даст 0 без ошибки — проверяйте `end`) и на числах за пределами `int8_t` (`128` превратится в −128).
- *Какую ошибку можно словить при дописывании в `digit_str`?* Переполнение буфера, если в файле длинное число. Надо проверять длину до записи.
- *Что забыли перед `fclose`?* Добавить последнее число из буфера.

Тесты на чтение со слайда:

Функциональные	Граничные
положительные `1 2 3 4 5`	файл не существует
отрицательные `-1 -2 -3`	пустой файл, файл из одних пробелов
смешанные `-1 1 -2 2 3`, нули `0 0 1 0 0`	`-128` и `127` — минимум и максимум
несколько пробелов подряд	`-129` и `128` — выход за пределы
чужие символы `a 1 2 3`	число из 5 и из 6 символов
ведущие нули `01 02 03`	10^4 чисел; 10^7 чисел (проверка на утечки)

Шаг 2. Сглаживание, версия 1 — на голых массивах. Валидация: окно не меньше 2 и меньше длины массива. Дальше для каждой позиции `i` копируем `window_size` элементов в массив-окно, считаем среднее, добавляем в результат через `realloc`.

Что плохо в этой версии (по словам лектора): надо вручную выделять и перевыделять память, отдельно хранить размер, и окно каждый раз заполняется заново. Это O(N × window_size).

Шаг 3. Выносим работу с памятью в `Vector`. Теперь `readArrayV2` и `smoothV2` просто вызывают `appendVectorItem`, а размер живёт внутри вектора.

Реализация Vector

```
typedef struct {
    void*  data;       // блок данных
    size_t elem_size;  // размер одного элемента
    size_t size;       // сколько элементов сейчас
    size_t capacity;   // на сколько выделена память
} Vector;
```

Каждая функция — несколько логических шагов:

1. **`createVector(elem_size)`**: `malloc` под саму структуру → проверить. `malloc(MIN_SIZE * elem_size)` под данные → проверить. Если не вышло, освободить структуру, чтобы не было утечки. Поставить `size = 0`, `capacity = MIN_SIZE`.
2. **`appendVectorItem(v, el)`**: проверить `v` и `el` на `NULL` → если места нет, увеличить capacity (`resize`) → скопировать элемент в позицию `size` → `size++`.
3. **`popVectorItem(v, index)`**: проверить индекс → скопировать удаляемый элемент в новую память (её потом освобождает тот, кто вызвал) → сдвинуть все элементы правее на один влево → `size--` → при необходимости уменьшить capacity.

void* — как сделать вектор для любого типа

Первая версия вектора хранила `int8_t*` и работала только с одним типом — это и есть «серьёзное ограничение» со слайда. Решение — `void*`.

`void*` — это адрес без типа. Разыменовать его нельзя (`*p = 5` — ошибка): компилятор не знает, сколько байт читать. Нужно сначала привести тип: `*(int*)p`. Вы уже пользовались `void*`, сами того не зная: `malloc` возвращает именно его.

Самый важный трюк лекции — как положить и достать i-й элемент, не зная его типа:

```
// положить value в позицию i
memcpy((char*)v->data + i * v->elem_size, &value, v->elem_size);

// достать адрес i-го элемента
void* item = (char*)v->data + i * v->elem_size;
int x = *(int*)item;   // вызывающий знает, что там int
```

Разбор по частям:

- `(char*)v->data` — приводим к `char*`, потому что `char` = 1 байт, и тогда `+ n` сдвигает ровно на n байт.
- `i * v->elem_size` — сдвиг в байтах. Для `int` и индекса 3 это 3 × 4 = 12 байт.
- `memcpy(куда, откуда, сколько байт)` — копирует сырые байты, ей всё равно, какой там тип.

`elem_size` для разных типов: `int` — 4, `short` — 2, `double` — 8, `char` — 1, `struct Point` — `sizeof(struct Point)`. Для строк на лекции пока договорились ограничиться 100 байтами.

Связный список

```
typedef struct Node {
    void* data;          // данные узла
    struct Node* next;   // следующий узел (NULL — конец)
} Node;

typedef struct {
    Node*  head;         // первый узел
    size_t elem_size;
} GenericList;
```

Функции: `createList`, `appendItem`, `findItem`, `popItem`, `listLength`, `freeList`. Обход списка всегда одинаковый: `Node* cur = list->head; while (cur) { ...; cur = cur->next; }`.

Указатель на функцию («лямбда» в C). Список не знает тип данных, поэтому не знает, как их сравнивать. Сравнение передают в `findItem` снаружи:

```
typedef int (*EqualsFunc)(const void* a, const void* b);  // тип «функция сравнения»

int intEquals(const void* a, const void* b) {
    return *(int*)a == *(int*)b;
}

findItem(list, &value, intEquals);  // передаём функцию как аргумент
```

Сглаживание, версия 3: список как окно

Окно сдвигается на один элемент за шаг: справа приходит новое число, слева уходит старое. Это очередь, а для неё список идеален: `append` в конец и `pop(0)` с начала — оба O(1). У массива `pop(0)` стоил бы O(N).

Идея для дальнейшего ускорения (на лекции её не было): храните сумму окна. При сдвиге прибавляйте новое число и вычитайте ушедшее, тогда среднее считается за O(1) вместо O(window_size), и всё сглаживание — за O(N).

Ловушки в коде со слайдов и шпаргалка

Код на слайдах — иллюстрация идеи, а не готовый код для копирования в лабу. В нём есть ошибки. Часть лектор оставил нарочно («остановиться и подумать!»), часть — опечатки. Их полезно знать: ровно такие же ошибки ловят на проверке лаб.

Где	Что не так	Как правильно
Л6, `read_array`	`if (strlen(digit_str) == ' ')` — длина сравнивается с кодом пробела (32)	`if (symbol == ' ')`
Л6, `read_array`	`*arr_size++` сдвигает сам указатель, а не увеличивает значение	`(*arr_size)++`
Л6, `read_array`, `smoothV1`	`arr = realloc(arr, ...); if (arr == NULL) free(arr);` — если realloc не смог, старый блок потерян, а `free(NULL)` ничего не делает	`tmp = realloc(arr, ...); if (!tmp) { free(arr); return NULL; } arr = tmp;`
Л6, `read_array`	если `fopen` вернул NULL, уже выделенный `arr` не освобождается	сначала открыть файл, потом выделять память, или `free(arr)` перед выходом
Л6, `read_array`	`char digit_str[5]` и проверка `len >= 5`: при `len = 4` запись `'\0'` идёт в `digit_str[5]` — за границу	проверять `len + 1 >= sizeof(digit_str)` до записи
Л6, `appendVectorItem` (версия для int8)	`vector->data[size] = el` — в число записывается адрес	`= *el` (а в универсальной версии — `memcpy`)

Где	Что не так	Как правильно
Л6, `smoothV2`	`window[j] = getVectorItem(...)` — функция возвращает указатель	`window[j] = *(int8_t*)getVectorItem(...)`
Л6, `findItem`	после `break` функция всё равно возвращает `-1`	`if (cmp(...)) return index;`
Л6, `smoothV3`	`pop(0)` делается до подсчёта среднего, поэтому в окне оказывается `window_size - 1` элементов	append → если длина > window_size, pop(0) → если длина == window_size, считать среднее
Л4, делители	для отрицательных `i * i == num` никогда не выполняется: для −100 получится 20 вместо 18	сравнивать с модулем: `i * i == abs(num)`
Л3, пароль	в варианте `while` переменная `input_sym` читается до инициализации и сравнивается с `NULL`	использовать `do-while` или сначала прочитать первый символ

Шпаргалка: вопросы для самопроверки

1. Какие 4 этапа у gcc и какие файлы на выходе? — Препроцессинг `.i` → компиляция `.s` → ассемблирование `.o` → линковка → исполняемый файл.
2. На каком этапе упадёт программа без `main`? — На линковке.
3. Чем статическая библиотека отличается от динамической? — Статическая вшивается в файл, динамическая подгружается при запуске.
4. Почему можно вернуть `"blue"` из функции, но нельзя вернуть локальный `char buf[]`? — Строковая константа живёт всю программу, локальный массив исчезает со стека.
5. Что напечатает `switch` без `break`? — Все case ниже совпавшего, до первого `break` или `return`.
6. `malloc` vs `calloc`? — calloc зануляет память, malloc оставляет мусор.

7. Чем функциональный тест отличается от граничного и стресс-теста? — Обычные входы / крайние случаи (0, пусто, максимум типа, NULL) / большие и худшие входы и время.
8. `sizeof('a')` и `sizeof("a")` в C? — 4 (литерал — int) и 2 (символ + `\0`).
9. `sizeof(struct { char c; double d; })`? — 16 из-за выравнивания.
10. Почему float нельзя сравнивать через `==`? — Хранится приближённо, мантисса всего 23 бита.
11. На сколько байт сдвигает `p + 1` для `int*`? — На 4 (на `sizeof(int)`).
12. Чем `size` отличается от `capacity`? — Сколько элементов есть / на сколько выделена память.
13. Почему `void*` нельзя разыменовать? — Компилятор не знает, сколько байт читать.
14. Зачем `(char*)` в `(char*)data + i * elem_size`? — Чтобы сдвиг считался в байтах.
15. Почему для скользящего окна список лучше массива? — `pop(0)` у списка O(1), у массива O(N).